

Modified YoloV10x for Object Detection on Drone-based Image Sensing

Rinta Kridalukmana

Department of Computer Engineering
Diponegoro University
Semarang, Indonesia
rintakrida@ce.undip.ac.id

*Dania Eridani

Department of Computer Engineering
Diponegoro University
Semarang, Indonesia
dania@ce.undip.ac.id
*Corresponding author

Risma Septiana

Department of Computer Engineering
Diponegoro University
Semarang, Indonesia
rismaseptiava@live.undip.ac.id

Ike Pertiwi Windasari

Department of Computer Engineering
Diponegoro University
Semarang, Indonesia
ike@ce.undip.ac.id

Abstract—Images captured by an AI-powered drone are often used for various purposes, including object detection. However, detecting small objects remains challenging in drone-captured images, requiring a reliable deep-learning architecture to handle feature extraction and image downsampling during the machine-learning process. This research aims to propose a deep-learning architecture to tackle such challenges. Based on the YoloV10x architecture, the conditional identity blocks in the backbone part are replaced by convolution and bottleneck blocks for faster and more efficient downsampling. Using the VISDRONE DET dataset, the learning process was performed from scratch. The model's performance evaluation uses mean Average Precision (mAP) and loss functions. Compared to the baseline architecture called Realtime Detection Transformer (RT-DETR), the proposed architecture can increase by approximately an average of 27% mAP scores for each class in the dataset.

Index Terms—deep learning, drone, object detection

I. INTRODUCTION

Data acquisition methods such as cameras and sensors have become essential in the smart city era. They are used to capture and analyze activities occurring in the concerned area and sometimes control some devices [1]. Even though theoretically, cameras and sensors can be installed everywhere, in some cases, there is a high demand for data acquisition tools that can be easily transported anywhere [2]. Such demand is to serve smart city apps that require a broader observation area to monitor activities and recognize occurrences or objects in some parts of the city [3]. Additionally, portable data acquisition tools help reach locations with limited access to power sources. In this regard, drones can be considered to support such flexibilities.

Undoubtedly, the most popular device used by drones is a camera, particularly for supporting monitoring tasks. Powered by artificial intelligence (AI) in the backend, the images captured by a drone are processed to recognize objects or

events on the ground for further analysis [4]. Consequently, generating a model for recognition tasks becomes an essential part, not only for object classification but also for object detection [5]. Previous studies showed that many researchers and practitioners have used the deep-learning-based method to develop such a model [6]. It is indicated that generating a recognition model using deep learning can achieve the desired accuracy even though sometimes it requires high computational costs in terms of processing units and training time [7].

The deep learning architecture can vary, and some researchers divide it into three parts: the backbone, neck, and head [8]. The backbone plays a significant role in extracting and encoding features from the input images, including low-level and high-level features. The neck part then enhances the extracted features, and the output is further processed by the head that performs the recognition tasks. Researchers have proposed various architectures. For example, You Only Look Once (YOLO) [9], ResNet [10], EfficientNet [11], Realtime Detection Transformer (RT-DETR) [12], and DarkNet [13]. However, as images captured by drones often consist of small objects, the accuracy of the recognition model becomes a challenge.

Previous studies in many fields have taken advantage of drone-captured images, which can also be considered remote-sensing images. For instance, Puspitasari et al. [14] generated a flood area segmentation model trained under EfficientNet architecture using the FloodNet dataset. The model was evaluated based on the accuracy and intersection over union (IOU) scores. Furthermore, Kumar et al. [15] conducted a benchmark study over U-Net, ResNet, DenseNet, and EfficientNet architectures, and their study showed that U-Net got the best scores based on IOU, validation loss, and Jacard coefficient after the model was trained using satellite image dataset. However, it should be noted that those researchers utilized pre-trained models. An example study that developed a recognizing model from scratch was

This research is funded by the Faculty of Engineering, Diponegoro University, through Strategic Research Program No. 81/UN7.F3/HK/IV/2024.

demonstrated by Huang et al. [16], who enhanced SegNet architecture to generate a model to detect crop planting areas from remote sensing images.

Even though a pre-trained model can boost accuracy and save computational costs, it has some drawbacks. For example, it may not be well-suited for specific purposes as the model might be trained using an irrelevant dataset, which can degrade its accuracy [17]. Hence, developing the recognition model from scratch is in high demand [18]–[20], and enhancing the deep learning architecture used to train the model becomes one option to improve the model's accuracy.

This research aims to develop a model to serve as an object detector on images captured by a drone. To create the model from scratch, a modified architecture of YOLOv10x is proposed. The modification, which can be considered our novelty, was made due to the conditional identity block's performance during the downsampling process. Those blocks were replaced with convolution and bottleneck blocks with a simpler architecture to boost the downsampling performance during feature extraction. Unlike [16] that relies on high-resolution segmentation maps, the proposed approach allows multi-scale representation for various abstraction levels. The results showed that the model generated with the proposed architecture has a better prediction performance than the baseline architecture, RT-DETR.

The remaining sections of this paper are organized as follows. Section 2 contains the research's experimental setup. Furthermore, the results and discussions are presented in Section 3. Finally, the conclusions are drawn in Section 4.

II. EXPERIMENTAL SETUP

This section details the experimental setup in this research, including the proposed deep learning architecture, training configurations, evaluation metrics, and the dataset. The proposed architecture is based on YOLOv10x architecture (Y10) [21]. The mean average precision (mAP) is the primary evaluation metric to measure performance against baseline architectures. At the end of the section, the VisDrone dataset is explained. This dataset will be used to train the proposed and baseline architecture from scratch.

A. Proposed Deep Learning Architecture

The proposed architecture is a modified Y10 (mY10) that receives three channels of input images with a size of 640×640 pixels. The input image is sent to the backbone. The backbone (see Fig. 1) is initialized with two convolutional layers (P1 and P2), each with three kernels, while their stride and padding are set to 2 and 1, respectively. The two convolutional layers have 64 and 128 channels, respectively.

The first downsampling process in the backbone is handled by three 128-channel C2f blocks, each of them comprises two 3×3 convolutions and three Bottleneck blocks for faster processing by applying a cross-stage partial connection (CSP) technique. After the first downsampling, one 3×3 convolutional layer (P3) is applied to continue the feature extraction. This time, the extraction uses 256 channels. Similar to previous convolutional layers, the kernel is set to three, stride = 2, and padding = 1. In all convolutional

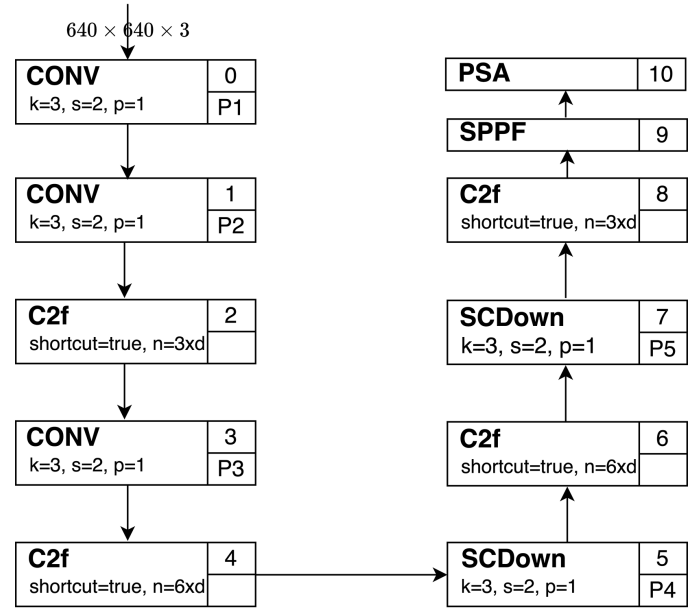


Fig. 1. mY10 backbone.

layers, the output is concatenated, and by default, they use Sigmoid Linear Unit (SiLU) for the activation function, that can be denoted as follows:

$$SiLU(x) = x \times sig(x) \quad (1)$$

where x is the input value and $sig(x)$ is the sigmoid function that normalizes any positive or negative input number to a value ranging from 0 to 1.

The second downsampling process involves three C2f blocks and two Spatial Channel Downsample (SCDown) modules (P4 and P5) arranged alternately. Such C2f blocks differ from the original Y10, which only has one C2f block. The other two C2f blocks are combined with the conditional identity block (CIB). Furthermore, the SCDown reduces channel and spatial dimensions. The input channel is mapped into a spatial structure by combining neighboring samples or pixels to remove noises, reduce the amount of data, and enhance resolutions.

The last two layers in the mY10's backbone are the Spatial Pyramid Pooling-Fast (SPPF) and Position-wise Spatial Attention (PSA) modules. All those modules have 1024 channels. SPPF allows a multi-scale representation of input features to capture different levels of feature abstraction. Such a representation is useful for detecting objects of various sizes. Meanwhile, PSA enhances the global representation of input features with minimal overhead.

After upsampling them using the nearest neighbor mode, the neck part of mY10 combines features extracted from the P3 and P4 backbone levels. The upsampled features are then concatenated and downsampled using C2f blocks with CIB. Such mixing feature operations are conducted in multiple scales with the support of the path aggregation network (PAN). The concatenation of the P3 backbone level becomes

8-pixel samples representing feature information for small-scale size dimensions, called 'P3 head'. Meanwhile, the P4 backbone level concatenation is considered a regression head to predict the bounding box coordinate of detected objects.

In addition to small dimensions, the head part generates medium-size (16-pixel) and large-size (32-pixel) dimensions based on features extracted and concatenated from a convolutional layer and a SCDowN layer, respectively. The former is called 'P4 head', while the latter is called 'P5 head'. In addition to small dimensions, the head part generates medium-size (16-pixel) and large-size (32-pixel) dimensions based on features extracted and concatenated from a convolutional layer and a SCDowN layer, respectively. The former is called 'P4 head', while the latter is called 'P5 head'. Finally, the classification head (detector block) uses the information from P3-P5 heads to identify the class of detected objects.

B. Hyperparameters

This research selected the Stochastic Gradient Descent (SGD) as the optimizer during the learning process. The SGD was chosen for its efficiency and simplicity, particularly in handling a large dataset. The learning rate was set to 0.01, and the momentum value was 0.9. There are three parameter groups: 221 weight has a decay value of 0, 234 weight has a decay value of 0.005, and 233 bias has a decay value of 0. The training epoch was set to 100 and the batch size of 4. A small batch size was selected to improve model accuracy. The input image size was 640.

C. Evaluation

The model's performance generated by the proposed mY10 deep learning architecture was evaluated against the baseline architecture called RT-DETR. The RT-DETR was selected because, according to Jun et al., RT-DETR beats YOLO in real-time object detection [12]. The evaluation used the mean average precision (mAP) matrices and several loss functions, including validation and box loss. The mAP is a widely used tool to calculate how effective the learning model is in recognizing and locating objects. The calculation is based on the average area computation of correct detection against the predicted total coverage area by the model.

In calculating mAP, the average precision (AP) should be determined by determining the overlap ratio given the correct and predicted bounding box in a certain number (n) of a query (q). Hence, mAP can be formulated as follows:

$$mAP = \frac{\sum_{q=1}^n AP(1)}{n} \quad (2)$$

Several widely used mAP thresholds, such as mAP50 and mAP75, are used. In this research, the performance measurement was only based on mAP50 values. This threshold was selected because it is the most popular threshold for examining recognition model performance.

D. Dataset

This research used the VISDRONE DET dataset containing images captured by a drone, which can be accessed at <https://github.com/VisDrone/VisDrone-Dataset> [22]. The

dataset has 8629 images and ten labels for the objects. The labels include pedestrian, people, bicycle, car, van, truck, tricycle, awning-tricycle, bus, and motor. The image annotations are stored using the YOLO text file format consisting the class index number and four bounding box coordinates. The annotation examples can be seen in Fig. 2.



Fig. 2. Dataset image annotation examples.

III. RESULTS AND DISCUSSIONS

This section presents the research results. First, a loss value analysis is provided, covering class loss, box loss, recall, and precision of the compared model. After that, the comparison of mAP scores with the threshold value of 50 is presented. Moreover, this section presents the discussions and potential works for future research.

A. Loss Value Analysis

Fig. 3 illustrates the movement of train class loss (TCL) and validation class loss (VCL) values between mY10 and RT-DETR during the 100-epoch training process. From the figure, it can be inferred that mY10's TCL and VCL values are never lower than RT-DETR's. Even at the beginning of the epoch, there is a significant gap. While the mY10's TCL value reached 19.116, the other is very low, 1.154. Similarly, the mY10 and RT-DETR's VCL are 9.3539 and 0.09954, respectively. However, at the end of the training, mY10's TCL and VCL values dropped significantly to 1.6114 and 1.8675. Although RT-DETR's TCL declined slightly to 0.53279, the VCL slightly decreased to 0.54732.

Based on the TCL and VCL, it can be highlighted that the mY10's TCL and VCL values have a more ideal class loss curve, showing a decreasing pattern. In the meantime, there was a symptom of RT-DETR having a problem in learning the dataset as their trends tended to increase, which

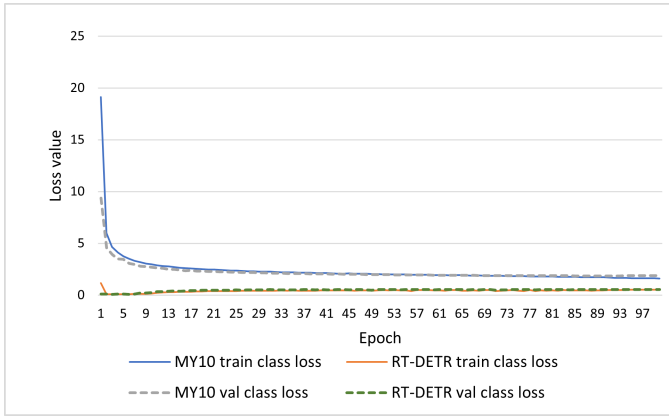


Fig. 3. Class loss during 100 epoch training process.

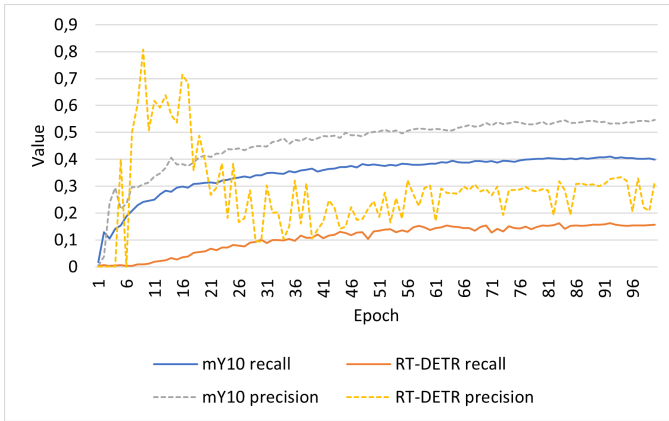


Fig. 4. Recall and precision during 100 epoch training process.

is abnormal. Such issues are also indicated by the recall and precision curves presented in Fig. 4. The RT-DETR's recall curve is very fluctuating. The possible reason is small object feature extraction, mainly the scaling images.

The mY10's recall and precision curves also have good shapes, indicated by the stable increasing patterns. Combining decreasing patterns from Fig. 3 and increasing patterns from Fig. 4, it can be inferred that mY10 generates a better model for the dataset. However, the mY10's class loss value is still higher than 1. The acceptable precision value for binary classification is 0.6. However, for multi-class problems like the case of this research, the precision value should be at least above 0.5, which failed to be achieved by RT-DETR.

Regarding box loss, RT-DETR got better scores as it is considerably lower than mY10. Fig. 5 shows that at the end of the epoch, the RT-DETR's train and validation box loss are 1.1412 and 1.0717, respectively. Meanwhile, the mY10s got 2.7003 and 2.8433, respectively. However, it should be noted that the previous loss values are also considered. Thus, based on those values, the mY10's model is more promising than the opponent's.

B. mAP Results

As seen in Table I, the result of the mAP comparison between mY10 and RT-DETR architectures showed that

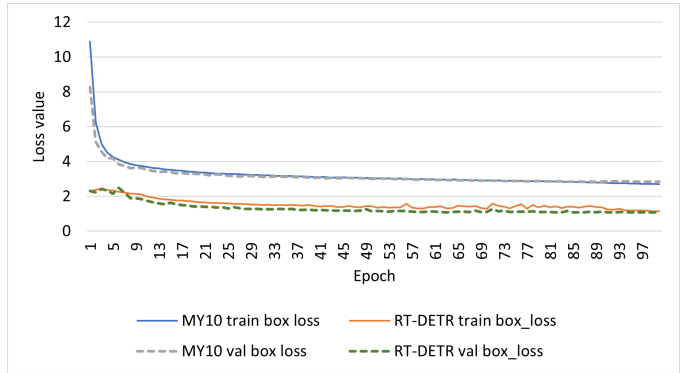


Fig. 5. Box loss and GIOU loss during 100 epoch training process.

from ten classes used in this research, all classes in mY10 showed a higher score than RT-DETR. The mY10 backbone architecture, used as the proposed method, had approximately 27% better scores in average than the baseline method. Thus, it can be inferred that the mAP scores support the loss function results.

TABLE I
MAP SCORES OF MY10 AND RT-DETR

Class	Images	Instances	mY10 mAP50	RT-DETR mAP50
Pedestrian	520	8844	0.434	0.153
People	482	5125	0.334	0.0149
Bicycle	364	1287	0.147	0.0027
Car	515	14064	0.811	0.635
Van	412	1975	0.486	0.156
Truck	266	750	0.421	0.056
Tricycle	337	1045	0.319	0.069
Awning-cycle	220	532	0.188	0.079
Bus	131	251	0.607	0.111
Motor	485	4886	0.468	0.159

C. Future Works

Even though the mY10 architecture outperformed RT-DETR in the study, several aspects can be enhanced for future research, including deep learning architecture and drone image-capturing strategy. Regarding deep learning architecture, future research can improve feature extraction techniques for small objects. Furthermore, image-capturing should consider exploiting the image stabilization technique, as the wind speed can shake the drone and produce blurry images.

IV. CONCLUSIONS

This paper proposes a deep learning architecture by modifying Y10. The main modification is replacing the conditional identity blocks in the backbone part with C2f blocks comprising convolutional and bottleneck blocks. The primary consideration of such replacements is simplifying the CIB's VGG architecture for a faster and more efficient downsampling process. The results show that the mAP50 scores of

the model generated by the proposed mY10 outperformed the ones generated by RT-DETR.

Future research in this area can focus on extracting features for small objects, which is essential for object detection in cases such as drone-captured images. Furthermore, tackling wind speed problems is also necessary during drone-image-capturing processes. In this regard, the image stabilization technique can be an option.

REFERENCES

- [1] B. Prabu, R. Malathy, M. N. A. G. Taj, and N. Madhan, "Drone networks and monitoring systems in smart cities," in *AI-Centric Smart City Ecosystems*, CRC Press, 2022, pp. 123–148.
- [2] B. Taha and A. Shoufan, "Machine learning-based drone detection and classification: State-of-the-art in research," *IEEE access*, vol. 7, pp. 138669–138682, 2019.
- [3] D. Tezza and M. Andujar, "The state-of-the-art of human–drone interaction: A survey," *IEEE Access*, vol. 7, pp. 167438–167454, 2019.
- [4] M. Elloumi, R. Dhaou, B. Escrig, H. Idoudi, and L. A. Saidane, "Monitoring road traffic with a UAV-based system," in 2018 IEEE wireless communications and networking conference (WCNC), 2018, pp. 1–6.
- [5] L. Wen et al., "Detection, Tracking, and Counting Meets Drones in Crowds: A Benchmark," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2021, pp. 7812–7821.
- [6] A. Saif and Z. R. Mahayuddin, "Crowd density estimation from autonomous drones using deep learning: challenges and applications," *Journal of Engineering and Science Research*, vol. 5, no. 6, pp. 1–6, 2021.
- [7] M. K  chhold, M. Simon, V. Eiselein, and T. Sikora, "Scale-adaptive real-time crowd detection and counting for drone images," in 2018 25th IEEE International Conference on Image Processing (ICIP), 2018, pp. 943–947.
- [8] P. Cai et al., "Detecting Individual Plants Infected with Pine Wilt Disease Using Drones and Satellite Imagery: A Case Study in Xianning, China," *Remote Sens (Basel)*, vol. 15, no. 10, 2023, doi: 10.3390/rs15102671.
- [9] H.-K. Jung and G.-S. Choi, "Improved YOLOv5: Efficient Object Detection Using Drone Images under Various Conditions," *Applied Sciences*, vol. 12, no. 14, 2022, doi: 10.3390/app12147255.
- [10] J. Gonzalez-Trejo and D. Mercado-Ravell, "Dense crowds detection and surveillance with drones using density maps," in 2020 International Conference on Unmanned Aircraft Systems (ICUAS), 2020, pp. 1460–1467.
- [11] M. Mahasin and I. A. Dewi, "Comparison of cspdarknet53, cspresnext-50, and efficientnet-b0 backbones on yolo v4 as object detector," *International Journal of Engineering, Science and Information Technology*, vol. 2, no. 3, pp. 64–72, 2022.
- [12] E. L. T. Jun, M.-L. Tham, and B.-H. Kwan, "A Comparative Analysis of RT-DETR and YOLOv8 for Urban Zone Aerial Object Detection," in 2024 IEEE International Conference on Automatic Control and Intelligent Systems (I2CACIS), 2024, pp. 340–345.
- [13] S. Albahli, N. Nida, A. Irtaza, M. H. Yousaf, and M. T. Mahmood, "Melanoma lesion detection and segmentation using YOLOv4-DarkNet and active contour," *IEEE access*, vol. 8, pp. 198403–198414, 2020.
- [14] R. D. I. Puspitasari, F. Q. Annisa, and D. Ariyanto, "Flooded Area Segmentation on Remote Sensing Image from Unmanned Aerial Vehicles (UAV) using DeepLabV3 and EfficientNet-B4 Model," in 2023 International Conference on Computer, Control, Informatics and its Applications (IC3INA), 2023, pp. 216–220. doi: 10.1109/IC3INA60834.2023.10285752.
- [15] D. G. Kumar and S. Chaudhari, "Comparison of Deep Learning Backbone Frameworks for Remote Sensing Image Classification," in *IGARSS 2022 - 2022 IEEE International Geoscience and Remote Sensing Symposium*, 2022, pp. 7763–7766. doi: 10.1109/IGARSS46834.2022.9883153.
- [16] Y. Huang, L. Tang, D. Jing, Z. Li, Y. Tian, and S. Zhou, "Research on Crop Planting Area Classification From Remote Sensing Image Based on Deep Learning," in 2019 IEEE International Conference on Signal, Information and Data Processing (ICSIDP), 2019, pp. 1–4. doi: 10.1109/ICSIDP47821.2019.9172915.
- [17] N. van Noord and E. Postma, "Learning scale-variant and scale-invariant features for deep image classification," *Pattern Recognition*, vol. 61, pp. 583–592, 2017, doi: <https://doi.org/10.1016/j.patcog.2016.06.005>.
- [18] A. Hussain, K. N. Qureshi, A. Aslam, T. Tariq, and M. R. Abdullahi, "TSC18 Convolutional Neural Network for Traffic Sign Classification," in 2024 4th International Conference on Emerging Smart Technologies and Applications (eSmarTA), 2024, pp. 1–6.
- [19] A. Hussain, K. N. Qureshi, R. W. Anwar, and A. Aslam, "A Novel SCD11 CNN Model Performance Evaluation with Inception V3, VGG16 and ResNet50 Using Surface Crack Dataset," in 2024 2nd International Conference on Unmanned Vehicle Systems-Oman (UVS), 2024, pp. 1–7.
- [20] A. Hussain, K. N. Qureshi, F. Zaman, A. Aslam, and others, "Minor Surface Cracks Detection using SCD11 Convolutional Neural Network," in 2024 4th International Conference on Emerging Smart Technologies and Applications (eSmarTA), 2024, pp. 1–7.
- [21] L. L. et al. Ao Wang Hui Chen, "YOLOv10: Real-Time End-to-End Object Detection," *arXiv preprint arXiv:2405.14458*, 2024.
- [22] P. Zhu et al., "Detection and tracking meet drones challenge," *IEEE Trans Pattern Anal Mach Intell*, vol. 44, no. 11, pp. 7380–7399, 2021.